# Reetrak Automation Framework

## Project Documentation and Maintenance Guide

Document purpose: This guide explains the Reetrak Automation project in an industry-standard format so that testers, automation engineers, leads, and new team members can understand the framework, run it correctly, extend it safely, and maintain it over time.

Document scope: This document covers the current framework implementation, its structure, usage model, execution flow, reporting, update process, and practical recommendations for future improvements.

---

## 1. Executive Summary

The Reetrak Automation project is a Selenium-based UI automation framework built for validating login and logout functionality of the Reetrak web application. It follows a Page Object Model structure and uses Java 21, Maven, TestNG, WebDriverManager, reusable utility classes, screenshot capture, and a custom HTML execution report.

At a high level, the framework is designed to:

* automate repetitive browser-based validation
* reduce manual effort in login and authentication testing
* support repeatable regression and smoke testing
* make test code easier to maintain as the application evolves
* provide execution evidence through screenshots and reports

This framework is already structured in a maintainable way, with clear separation between configuration, driver setup, page interactions, test cases, and reporting. That makes it a solid base for scaling into a broader enterprise automation suite.

---

## 2. Project Objective

The primary objective of this project is to validate critical authentication-related user journeys in the Reetrak application through automated browser-based testing.

The current implementation focuses on:

* successful login
* logout verification
* validation failures for blank credentials
* invalid username handling
* invalid password handling
* deactivated user handling
* locked user handling

This project can be used as:

* a smoke suite for authentication checks
* a regression suite for login-related functionality
* a starter framework for broader UI automation
* a reusable baseline for future automation modules

## 3. Why This Framework Exists

Manual UI testing is slow, repetitive, and error-prone, especially for core flows such as login and logout that must be checked frequently. This framework exists to solve that problem by automating predictable validation scenarios with reliable setup, reusable components, and standardized reporting.

### Business reasons

* authentication is a critical gateway flow
* failures in login/logout can block all other application access
* regression verification is needed after every release or bug fix
* consistent evidence is required for QA signoff

### Engineering reasons

* reusable components reduce duplication
* Page Object Model improves maintainability
* Maven standardizes build and execution
* TestNG provides controlled execution and assertions
* screenshots and reports improve debugging and traceability

## 4. Technology Stack and Why Each Tool Is Used

### Java 21

Java is the implementation language for the framework.

Why it is used:

* strong ecosystem for Selenium automation
* excellent compatibility with enterprise tooling
* stable object-oriented design for framework development
* widely adopted by QA and automation teams

When to use:

* when building scalable automation frameworks
* when integration with CI/CD and enterprise reporting is expected

How it helps here:

* provides the structure for page classes, utilities, and tests
* supports clean package-level separation of concerns

### Maven

Maven is used for dependency management and build execution.

Why it is used:

* standard project structure
* easy dependency version control
* repeatable builds

* easy command-line execution

When to use:

* when the team needs a standard Java build system
* when dependency management should remain centralized

How it helps here:

* controls library versions through ‘pom.xml‘
* uses Surefire to execute TestNG through Maven

## Selenium WebDriver

Selenium drives the browser and performs user-level interactions.

Why it is used:

* supports real browser automation
* suitable for validating end-to-end UI workflows
* works with major browsers

When to use:

* when the objective is to validate visible browser behavior
* when user interactions and page rendering matter

How it helps here:

* enters credentials
* clicks login and logout controls
* reads UI messages and page text

## TestNG

TestNG manages test execution, lifecycle hooks, priorities, assertions, and listeners.

Why it is used:

* strong test annotation support
* suite execution through XML
* native listener integration
* simple assertion model

When to use:

* when controlled execution order matters
* when reporting or custom listeners are needed

How it helps here:

* runs test cases in defined order
* manages setup and teardown
* integrates with the custom HTML report

## WebDriverManager

WebDriverManager automatically manages browser driver binaries.

Why it is used:

* removes manual driver setup effort
* reduces environment-specific driver issues

When to use:

* when local and CI environments need simplified browser startup

How it helps here:

* automatically resolves browser drivers for Chrome, Firefox, and Edge

## Page Object Model

Page Object Model is the design pattern used for page interaction classes.

Why it is used:

* avoids writing locators directly inside tests
* improves maintainability when UI changes
* keeps tests readable

When to use:

* when the automation suite has reusable screens or workflows

How it helps here:

* ‘LoginPage.java‘ contains login-specific element locators and methods

* tests remain focused on validation logic instead of UI interaction details

## 5. Current Project Architecture

The framework follows a layered structure.

### Layer 1: Configuration layer

Responsible for:

* environment values
* browser selection
* base URL
* test credentials and test data

Implemented through:

* ‘src/test/resources/config.properties‘
* ‘src/main/java/com/reetrak/automation/utilities/ConfigReader.java‘

### Layer 2: Driver management layer

Responsible for:

* browser creation
* browser selection
* timeout configuration
* driver cleanup

Implemented through:

* ‘src/main/java/com/reetrak/automation/factory/DriverFactory.java‘

### Layer 3: Reusable page interaction layer

Responsible for:

* common click, type, and read actions
* page initialization
* synchronization support through waits

Implemented through:

* ‘src/main/java/com/reetrak/automation/pages/BasePage.java‘
* ‘src/main/java/com/reetrak/automation/utilities/WaitUtils.java‘

## Layer 4: Business page layer

Responsible for:

* screen-specific locators
* screen-specific workflow methods
* reusable actions for the Reetrak UI

Implemented through:

* ‘src/main/java/com/reetrak/automation/pages/LoginPage.java‘

## Layer 5: Test execution layer

Responsible for:

* declaring test scenarios
* calling page methods
* validating expected outcomes

Implemented through:

* ‘src/test/java/com/reetrak/automation/base/BaseTest.java‘
* ‘src/test/java/com/reetrak/automation/tests/LoginLogoutTest.java‘

## Layer 6: Reporting and evidence layer

Responsible for:

* screenshots
* execution summaries
* traceability of test results

Implemented through:

* ‘src/main/java/com/reetrak/automation/utilities/ScreenshotUtils.java‘
* ‘src/test/java/com/reetrak/automation/reports/HtmlSummaryReporter.java‘

## 6. Detailed Project Structure

```
Reetrak Automation/
|-- pom.xml
|-- testng.xml
|-- docs/
|   |-- Reetrak_Automation_Project_Guide.md
|   |-- Reetrak_Automation_Project_Guide.pdf
|   ‘-- Generate-GuidePdf.ps1
|-- src/
|   |-- main/
|   |   ‘-- java/com/reetrak/automation/
|   |       |-- factory/
|   |       |   ‘-- DriverFactory.java
|   |       |-- pages/
|   |       |   |-- BasePage.java
|   |       |   ‘-- LoginPage.java
|   |       ‘-- utilities/
|   |           |-- ConfigReader.java
```

```
|   |           |-- ScreenshotUtils.java
|   |           ‘-- WaitUtils.java
|   ‘-- test/
|       |-- java/com/reetrak/automation/
|       |   |-- base/
|       |   |   ‘-- BaseTest.java
|       |   |-- reports/
|       |   |   ‘-- HtmlSummaryReporter.java
|       |   ‘-- tests/
|       |       ‘-- LoginLogoutTest.java
|       ‘-- resources/
|           ‘-- config.properties
|-- test-output/
|   |-- html-summary-report.html
|   ‘-- screenshots/
‘-- target/
```

## Structure explanation

‘src/main/java‘

* contains framework code and reusable automation components

‘src/test/java‘

* contains test classes, lifecycle handling, and reporting classes

‘src/test/resources‘

* contains configurable execution data such as browser, URL, and credentials

‘test-output‘

* contains generated reports and screenshots after execution

‘docs‘

* contains project-level documentation and the PDF export utility

---

## 7. End-to-End Execution Flow

This section explains what happens when the framework is executed.

### Step 1: Maven starts the test run

Command:

```
mvn clean test
```

Maven reads ‘pom.xml‘, resolves dependencies, compiles the project, and starts the Surefire plugin.

### Step 2: Surefire loads TestNG

The Surefire plugin uses:

* ‘testng.xml‘

This file tells TestNG:

* which test classes to run
* which listeners to attach

### Step 3: TestNG initializes the suite

The suite currently:

* runs with ‘parallel="false"‘
* registers ‘HtmlSummaryReporter‘
* executes the ‘LoginLogoutTest‘ class

### Step 4: ‘BaseTest‘ setup executes before each test

Before every test method:

* browser value is loaded
* base URL is loaded
* test data is loaded
* driver is created
* browser is opened and prepared

### Step 5: Test method executes

The test:

* creates the page object
* opens the application URL
* performs the user actions
* validates expected results using assertions

### Step 6: Teardown executes after each test

After every test:

* screenshot is captured for pass or fail
* screenshot path is attached to the result
* browser session is closed

### Step 7: HTML summary report is generated

After suite completion:

* execution metrics are collected
* rows are added for passed, failed, and skipped tests
* a styled HTML summary report is written to ‘test-output/html-summary-report.html‘

# 8. Key Files and Their Functional Role

## ‘pom.xml‘

Purpose:

* project metadata
* dependency management
* compiler configuration
* Surefire test execution setup

Why it matters:

* this is the build backbone of the framework
* dependency updates happen here
* Java version control happens here

Current notable dependencies:

* ‘selenium-java‘
* ‘testng‘
* ‘webdrivermanager‘
* ‘commons-io‘
* ‘slf4j-simple‘
* ‘scala-library‘

Current plugins:

* ‘maven-compiler-plugin‘
* ‘maven-surefire-plugin‘

## ‘testng.xml‘

Purpose:

* defines which classes are executed
* registers custom listeners
* controls suite-level behavior

Why it matters:

* it is the execution contract for TestNG
* new classes can be added here for suite inclusion

## ‘ConfigReader.java‘

Purpose:

* loads properties from ‘config.properties‘
* allows JVM system properties to override file values
* reads all ‘testdata.*‘ entries into a map

Why it matters:

* centralizes configuration access
* supports environment-specific execution

## ‘DriverFactory.java‘

Purpose:

* initializes browser drivers
* manages browser-specific startup
* maximizes the browser
* sets timeouts
* closes and removes the driver after execution

Why it matters:

* keeps driver lifecycle consistent
* avoids duplicate setup logic in tests

Current supported browsers:

* Chrome

* Firefox
* Edge

## ‘BasePage.java‘

Purpose:

* provides reusable page methods
* initializes with ‘PageFactory‘
* integrates waiting behavior with page interactions

Why it matters:

* keeps common UI actions centralized
* prevents repeated click/type/read logic across pages

## ‘LoginPage.java‘

Purpose:

* stores login and logout locators
* performs login workflow
* performs logout workflow
* reads validation and page messages

Why it matters:

* this is the current business page model of the application
* it shows how future pages should be created

## ‘BaseTest.java‘

Purpose:

* provides setup and teardown
* initializes ‘driver‘, ‘baseUrl‘, and test data

* captures screenshots

Why it matters:

* every test class should inherit consistent behavior

## ‘LoginLogoutTest.java‘

Purpose:

* contains current test scenarios
* validates actual application behavior

Why it matters:

* this is the functional coverage entry point of the existing framework

## ‘ScreenshotUtils.java‘

Purpose:

* captures screenshots
* stores them in pass or fail folders

Why it matters:

* gives execution evidence for debugging and reporting

## ‘HtmlSummaryReporter.java‘

Purpose:

* creates the post-run summary report
* aggregates execution results
* links screenshots to the report

Why it matters:

* gives an immediate business-friendly execution overview

## 9. Current Configuration and Test Data Model

Current configuration file:

‘src/test/resources/config.properties‘

Current values include:

* browser selection
* application URL
* valid username and password
* invalid username
* deactivated username
* invalid password

Current property examples:

```
browser=chrome

base.url=http://localhost:8080/TrackAndTraceServer/reeclient/#/pages/login

testdata.username=Tadmin

testdata.password=password

testdata.invalid.username=InvalidUser

testdata.deactive.username=jadmin

testdata.invalid.password=InvalidPassword
```

### Why this model is useful

* keeps environment-specific values outside the code
* supports reusability across tests
* allows quick updates without changing Java classes

### When to update this file

* when application URL changes
* when local environment changes
* when credentials change
* when additional test inputs are needed

### How overrides work

System properties take priority over file values.

Example:

```
mvn clean test -Dbrowser=edge -Dbase.url=http://new-env/app
```

This makes the framework suitable for both local runs and CI/CD execution.

---

## 10. Current Test Coverage

The current test suite contains seven scenarios.

### 1. Blank username and password

Purpose:

* validates required field behavior

Expected result:

* username validation message should appear
* password validation message should appear

### 2. Invalid username with valid password

Purpose:

* validates unknown-user rejection

Expected result:

* error message should state that the user does not exist

### 3. Valid username with invalid password

Purpose:

* validates password mismatch handling

Expected result:

* user should remain on the login page

### 4. Valid username and password

Purpose:

* validates successful login

Expected result:

* dashboard title should be visible

### 5. Logout verification

Purpose:

* validates exit flow from an authenticated session

Expected result:

* application should return to the login page

### 6. Deactivated user verification

Purpose:

* validates inactive account behavior

Expected result:

* deactivated-user message should be shown

### 7. Locked user verification

Purpose:

* validates lock behavior after repeated invalid attempts

Expected result:

* locked-user message should be shown

---

## 11. How and When to Use This Framework

### Recommended usage scenarios

Use this framework:

* before production releases
* after authentication-related code changes
* after bug fixes in login/logout modules
* as part of daily smoke validation
* during regression cycles
* in QA, UAT, or staging validation pipelines

### When this framework is the right choice

Use UI automation when:

* business wants validation from an end-user perspective
* visual and interactive behavior must be confirmed

* login flow includes page rendering and UI messaging that API tests cannot fully cover

### When not to rely on it alone

Do not use UI automation as the only validation layer when:

* API tests can validate the logic faster
* the feature is highly unstable
* test execution speed is the main priority

### Balanced testing strategy

Industry standard practice is:

* unit tests for code-level validation
* API tests for business rules and service contracts
* UI tests for complete user journeys and critical flows

This framework belongs to the UI automation layer of that strategy.

---

## 12. How to Run the Framework

## Prerequisites

Before execution, ensure the following are available:

* Java 21 installed and configured
* Maven installed and available in PATH
* at least one supported browser installed
* target application URL reachable

## Basic execution

```
mvn clean test
```

## Run on a specific browser

```
mvn clean test -Dbrowser=chrome

mvn clean test -Dbrowser=firefox

mvn clean test -Dbrowser=edge
```

## Run on a specific environment URL

```
mvn clean test -Dbase.url=http://your-server/app
```

## Run with different credentials

```
mvn clean test -Dtestdata.username=MyUser -Dtestdata.password=MyPassword
```

## Expected execution artifacts

After execution, review:

* ‘test-output/html-summary-report.html‘
* ‘test-output/screenshots/pass‘
* ‘test-output/screenshots/fail‘

# 13. How to Add New Test Cases

This section explains the standard way to expand the suite without breaking framework design.

## Standard approach

1. identify the business scenario
2. identify which page or component is involved
3. create or update page object methods
4. add a new test method or test class
5. use assertions in the test layer
6. add test data if needed
7. run the suite and review the report

## Example: add a forgot-password test

Add locator and action in ‘LoginPage.java‘:

```
@FindBy(xpath = "//a[text()=’Forgot Password?’]")

private WebElement forgotPasswordLink;



public void openForgotPassword() {

    click(forgotPasswordLink);

}
```

Add a test:

```
@Test

public void forgotPasswordPageShouldOpen() {
```

```
    LoginPage loginPage = new LoginPage(driver);

    loginPage.open(baseUrl);

    loginPage.openForgotPassword();

    Assert.assertTrue(driver.getCurrentUrl().contains("forgot-password"));

}
```

### Best-practice rules while adding tests

* keep locators out of test classes
* keep assertions in test classes
* keep reusable interactions in page classes
* keep test data in configuration or a dedicated data source
* keep setup and teardown in the base layer

---

## 14. How to Add a New Page Object

Create a new page class when:

* the application navigates to a different screen
* a page has its own locators and actions
* multiple tests need the same page behavior

### Location

```
src/main/java/com/reetrak/automation/pages/
```

### Template

```
package com.reetrak.automation.pages;
```

```
import org.openqa.selenium.WebDriver;

import org.openqa.selenium.WebElement;

import org.openqa.selenium.support.FindBy;


public class DashboardPage extends BasePage {


    @FindBy(xpath = "//span[contains(normalize-space(), 'Dashboard')]")

    private WebElement dashboardTitle;


    public DashboardPage(WebDriver driver) {

        super(driver);

    }


    public String getDashboardTitle() {

        return getText(dashboardTitle);

    }

}
```

## Industry-standard guidance

* one class per logical page or component
* use meaningful method names
* avoid writing assertions inside page classes
* expose behavior, not raw locator handling

# 15. How to Add a New Test Class

Create a new class under:

```
src/test/java/com/reetrak/automation/tests/
```

### Template

```
package com.reetrak.automation.tests;


import com.reetrak.automation.base.BaseTest;
import com.reetrak.automation.pages.LoginPage;
import org.testng.Assert;
import org.testng.annotations.Test;


public class ExampleTest extends BaseTest {


    @Test
    public void sampleTest() {
        LoginPage loginPage = new LoginPage(driver);
        loginPage.open(baseUrl);
        Assert.assertTrue(true);
    }
}
```

### After creating the class

* add it to ‘testng.xml‘ if it should run in the suite
* execute the suite
* verify that screenshots and reports are generated

## 16. How to Update Existing Tests

Update test code when:

* the UI locator changes
* the expected behavior changes
* the screen flow changes
* the business rule changes

### Standard update sequence

1. identify whether the issue is locator-related or expectation-related
2. update the page object first if the element changed
3. update the assertion if the business outcome changed
4. update test data if needed
5. rerun tests and review evidence

### Examples

* update ‘@FindBy‘ if a button id changes
* update assertion text if the validation message changes
* add a new method if the workflow now includes extra steps such as OTP

## 17. How to Update Configuration

Configuration changes are usually made in:

‘src/test/resources/config.properties‘

### Update this file when

* browser defaults change
* environment URL changes
* test data changes
* new credentials or environment keys are required

### Sample update

```
browser=edge

base.url=http://new-server/app

testdata.username=NewAdmin

testdata.password=NewPassword
```

### Recommendation

Use file updates for stable local defaults. Use command-line overrides for temporary execution or CI/CD.

---

## 18. How to Update Dependencies

Dependency versions are managed from ‘pom.xml‘.

### Current version properties

* ‘selenium.version‘
* ‘testng.version‘
* ‘webdrivermanager.version‘
* ‘commons-io.version‘
* ‘slf4j.version‘
* ‘surefire.version‘
* ‘compiler.plugin.version‘

### Update process

1. open ‘pom.xml‘
2. change the required version property
3. run the suite
4. review compatibility issues if they appear

### Verification command

```
mvn clean test
```

### Dependency update cautions

* browser behavior can change after Selenium upgrades
* plugins can affect test execution flow
* driver compatibility should be revalidated

---

## 19. Reporting and Evidence Management

### Screenshot handling

‘ScreenshotUtils.java‘ stores screenshots in:

```
test-output/screenshots/pass
```

```
test-output/screenshots/fail
```

Why it matters:

* supports defect analysis
* preserves visual proof of the UI state
* helps understand pass and fail conditions

### HTML summary reporting

‘HtmlSummaryReporter.java‘ generates:

```
test-output/html-summary-report.html
```

Current report includes:

* total test count
* passed test count
* failed test count
* skipped test count
* execution duration
* screenshot links
* error messages
* class and method names

### How teams should use the report

* review failures first
* open linked screenshots
* confirm whether the failure is environment, locator, or product related
* archive the report as an execution artifact in CI/CD

## 20. Strengths of the Current Framework

The current implementation already includes several good framework practices.

* clean project organization
* separated page and test layers
* centralized driver setup
* reusable configuration reader
* multi-browser support
* custom report generation
* screenshot-based execution evidence
* reusable base test lifecycle

These strengths make the framework suitable for controlled expansion.

---

## 21. Current Gaps, Risks, and Improvement Opportunities

The framework is functional, but there are also areas that should be improved over time.

### Use of ‘Thread.sleep()‘

Why it is a risk:

* increases execution time
* makes tests brittle
* causes avoidable flakiness

Recommended improvement:

* replace hard waits with explicit waits in ‘WaitUtils‘

### Credentials stored in plain text

Why it is a risk:

* insecure for shared or production-like environments

Recommended improvement:

* move sensitive values to environment variables or CI secret storage

### Login and post-login actions are in one page class

Why it is a risk:

* class responsibility will grow too large as the application expands

Recommended improvement:

* split into dedicated page and component classes where needed

### Exact text assertions only

Why it is a risk:

* even small message wording changes can break tests

Recommended improvement:

* combine message validation with state or navigation validation where possible

### Single-suite scope

Why it is a risk:

* execution flexibility is limited as the suite grows

Recommended improvement:

* create separate suites such as smoke, regression, and module-specific collections

---

## 22. Recommended Maintenance Standards

To keep the framework maintainable, follow these standards.

### Coding standards

* keep page methods business-readable

* keep test methods scenario-focused
* use meaningful method names
* avoid duplicate locator logic

### Design standards

* pages should represent pages or reusable UI components
* tests should validate behavior, not manage low-level interaction details
* configuration should remain externalized

### Execution standards

* run the suite after every meaningful change
* review report and screenshots, not just pass/fail count
* keep environment assumptions documented

### Review standards

* verify locator stability
* check whether waits are appropriate
* confirm that assertions reflect business requirements

## 23. Suggested Future Roadmap

These enhancements would move the project closer to a mature enterprise automation framework.

* replace ‘Thread.sleep()‘ with condition-based waits
* add headless execution support
* add environment profiles such as dev, qa, uat, and prod-like
* add data-driven testing using ‘@DataProvider‘
* add tagging and grouping through TestNG
* split page responsibilities into more granular page/component objects
* strengthen secret management
* integrate with CI/CD artifact publishing

* add richer reporting if needed

# 24. Quick Reference Guide

### To add a new test

1. extend ‘BaseTest‘
2. create or reuse a page object
3. add page actions
4. add assertions in the test
5. update ‘testng.xml‘ if required
6. run the suite

### To update an existing flow

1. update the page object locator or method
2. update the test assertion if expected behavior changed
3. update test data or config if required
4. rerun and review reports

### To update config

1. edit ‘config.properties‘
2. or override values through ‘-D‘ system properties

### To update dependencies

1. modify version properties in ‘pom.xml‘
2. run ‘mvn clean test‘
3. validate compatibility

## 25. File Reference Index

* ‘pom.xml‘: build, dependencies, compiler, and test execution configuration
* ‘testng.xml‘: suite definition and listener registration
* ‘src/test/resources/config.properties‘: execution defaults and test data
* ‘src/main/java/com/reetrak/automation/utilities/ConfigReader.java‘: configuration access
* ‘src/main/java/com/reetrak/automation/factory/DriverFactory.java‘: browser lifecycle management
* ‘src/main/java/com/reetrak/automation/pages/BasePage.java‘: shared page actions
* ‘src/main/java/com/reetrak/automation/pages/LoginPage.java‘: login/logout page logic
* ‘src/main/java/com/reetrak/automation/utilities/WaitUtils.java‘: explicit waits
* ‘src/main/java/com/reetrak/automation/utilities/ScreenshotUtils.java‘: screenshot capture
* ‘src/test/java/com/reetrak/automation/base/BaseTest.java‘: setup and teardown lifecycle
* ‘src/test/java/com/reetrak/automation/tests/LoginLogoutTest.java‘: scenario coverage
* ‘src/test/java/com/reetrak/automation/reports/HtmlSummaryReporter.java‘: custom HTML reporting

## 26. Final Conclusion

The Reetrak Automation project is a well-structured Selenium TestNG framework designed to automate authentication-related validation for the Reetrak web application. It already includes strong foundational practices such as Page Object Model design, centralized driver setup, externalized configuration, evidence capture, and custom reporting.

This framework should be used whenever browser-level validation of login and logout flows is needed. It should be extended by adding well-structured page objects and test classes. It should be maintained by updating locators, configuration, and dependencies in their proper layers instead of mixing concerns.

Most importantly, the framework is not only useful for current login coverage. It is also a scalable starting point for future UI automation across the wider application. With continued improvement in waits, configuration security, suite organization, and reporting maturity, it can evolve into a strong enterprise-grade automation foundation without changing its core design philosophy.